

```
// INTERFACE LOGIC. One tap = one  
turn: listen -> think -> speak.  
// It knows nothing about HTML;  
app.js gives it a small "ui" object.
```

```
import { friendlyMessage } from  
"./errors.js";
```

```
export class Controller {  
  // ui = { setState(state, text,  
  isError), showYou(text),  
  showJarvis(text) }  
  constructor({ listener, speaker,  
  getBrain, ui, log = console }) {  
    Object.assign(this, { listener,  
    speaker, getBrain, ui, log });  
    this.state = "idle";  
    this.abortCtl = null;  
  }  
  
  _set(state, text = "", isError =  
false) {  
    this.state = state;  
    this.ui.setState(state, text,  
isError);  
  }  
}
```

```
// The big button. What it does  
depends on what JARVIS is doing right  
now.
```

```
press() {  
  switch (this.state) {  
    case "idle": return  
this._runTurn();  
    case "listening":  
this.listener.stop(); break;  
    case "thinking": if  
(this.abortCtl)  
this.abortCtl.abort(); break;  
    case "speaking":  
this.speaker.stop(); break;  
    default: break;  
  }  
  return Promise.resolve();  
}
```

```
async _runTurn() {  
  this.speaker.unlock();  
  this._set("listening");  
  try {  
    const heard = await  
this.listener.listen({ onInterim: (t)  
=> this.ui.showYou(t) });  
    this.ui.showYou(heard);  
    this._set("thinking");  
  }  
}
```

```
        this.abortCtl = new
AbortController();
        const reply = await
this.getBrain().respond(heard, {
signal: this.abortCtl.signal });
        this.ui.showJarvis(reply);
        this._set("speaking");
        await
this.speaker.speak(reply);
        this._set("idle");
    } catch (err) {
        this.log.error("[JARVIS]",
err);
        if (err && err.code ===
"aborted") this._set("idle",
"Cancelled.");
        else this._set("idle",
friendlyMessage(err), true);
    } finally {
        this.abortCtl = null;
    }
}
}
```